Table of contents

Series 3

Methods

Weakest Precondition (WP) Calculation

- **Method 1: WP for Assignment**
  For an assignment `x := e`, substitute variable `x` with expression `e` in postcondition ψ:

$$\mathrm{WP}(x := e, \psi) = \psi[x \leftarrow e]$$

- **Method 2: WP for Conditional**
  For an instruction `if B then c_1 else c_2`, combine the WP of both branches, conditioned by `B` and `¬B`:

$$\mathrm{WP}(\text{if } B \text{ then } c_1 \text{ else } c_2, \psi) = (B \Rightarrow \mathrm{WP}(c_1, \psi)) \wedge (\neg B \Rightarrow \mathrm{WP}(c_2, \psi))$$

- **Method 3: Consideration of Safety Conditions**
  When the program contains a partial operation (e.g., division), safety conditions (e.g., denominator non-zero) must be explicitly added to the WP.

Hoare Triple Verification

- **Method 4: Validity via Logical Implication**
  A triple $\{P\}\ c\ \{Q\}$ is valid if and only if the precondition P implies the WP of program c for postcondition Q:

  $$\{P\}\ c\ \{Q\} \text{ is valid} \quad \text{iff} \quad P \Rightarrow \mathrm{WP}(c, Q) \text{ is true.}$$

  All safety conditions (e.g., non-division by zero) must also be verified.

- **Method 5: Refinement with Additional Precondition**
  If the computed WP is not automatically verified, we can propose a stronger precondition P that makes the implication valid.

Structural Proofs on WP

- **Method 6: Structural Induction on Programs**

  To prove a property about WP for all programs, proceed by induction on the program structure:

  1. **Base case**: elementary instructions (assignment).

  2. **Induction steps**: sequential composition, conditional, loop.

Logical Manipulations

- **Method 7: Common Logical Transformations**
  - Implication elimination: $A \Rightarrow B \equiv \neg A \vee B$
  - De Morgan's laws: $\neg(A \wedge B) \equiv \neg A \vee \neg B$, $\neg(A \vee B) \equiv \neg A \wedge \neg B$
  - Distributivity: $A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$

Reminder

1. **WP Rules**
   - $\mathrm{WP}(x := e, \psi) = \psi[x \leftarrow e]$
   - $\mathrm{WP}(\text{if } B \text{ then } c_1 \text{ else } c_2, \psi) = (B \Rightarrow \mathrm{WP}(c_1, \psi)) \wedge (\neg B \Rightarrow \mathrm{WP}(c_2, \psi))$
2. **Hoare Triple Validity**

$$\{P\}\ c\ \{Q\} \text{ valid} \iff P \Rightarrow \mathrm{WP}(c, Q)$$

3. **Safety Conditions**
   For any division `a/b`, add the condition $b \neq 0$ to the WP.
4. **Useful Logic Laws**
   - Implication: $A \Rightarrow B \equiv \neg A \vee B$
   - Distributivity of $\wedge$ over $\vee$ and vice versa
   - Equivalence: $(A \Rightarrow (B \wedge C)) \equiv (A \Rightarrow B) \wedge (A \Rightarrow C)$
5. **Structural Induction**
   - Base: verify the property for simple instructions.
   - Induction: assume the property for subprograms, prove it for compositions.

Illustrative Examples

Example 1: WP Calculation for a Conditional (Methods 1 and 2)

Context: Compute $\mathrm{WP}(f, \psi_1)$ for the program

$$f(a, b) = \text{if } (a > 2) \text{ then } res := a \text{ else } res := b \text{ endif}$$

with $\psi_1 : res \geq 0$.

Application:

1. Apply the conditional rule:

$$\mathrm{WP}(f, \psi_1) = (a > 2 \Rightarrow \mathrm{WP}(res := a, \psi_1)) \wedge (\neg(a > 2) \Rightarrow \mathrm{WP}(res := b, \psi_1))$$

2. Compute the WP of assignments (Method 1):
   - $\mathrm{WP}(res := a, \psi_1) = \psi_1[res \leftarrow a] = a \geq 0$
   - $\mathrm{WP}(res := b, \psi_1) = \psi_1[res \leftarrow b] = b \geq 0$

3. Combine:

$$\mathrm{WP}(f, \psi_1) = (a > 2 \Rightarrow a \geq 0) \wedge (\neg(a > 2) \Rightarrow b \geq 0)$$

   i.e.,

$$(a > 2 \Rightarrow a \geq 0) \wedge (a \leq 2 \Rightarrow b \geq 0).$$

Example 2: Verification of Hoare Triple Validity (Methods 4 and 5)

Context: Verify if $\{P_2\}\ f\ \{\psi_2\}$ is valid, with $P_2 := \mathbf{T}$ (true) and $\psi_2 : res \geq 42$.

Application:

1. We already computed:

$$\mathrm{WP}(f, \psi_2) = (a > 2 \Rightarrow a \geq 42) \wedge (\neg(a > 2) \Rightarrow b \geq 42)$$

2. For validity, we need $P_2 \Rightarrow \mathrm{WP}(f, \psi_2)$, i.e., $\mathrm{WP}(f, \psi_2)$ must be true for all a, b.

3. However, if $a = 3$ (so $a > 2$ true), we must have $a \geq 42$, which is false. Therefore the implication is not true for all values.

4. **Conclusion**: The triple is not valid. However, it is *satisfiable* (e.g., with $a = 42, b = 42$).

Example 3: Consideration of Division by Zero (Methods 3 and 4)

Context: Compute $\mathrm{WP}(g, \psi_3)$ for

$$g(a, b) = \text{if } (a \geq b) \text{ then } res := a/b \text{ else } res := b/a \text{ endif}$$

with $\psi_3 : res \geq 1$.

Application:

1. Apply the conditional rule:

$$\mathrm{WP}(g, \psi_3) = (a \geq b \Rightarrow \mathrm{WP}(res := a/b, \psi_3)) \wedge (a < b \Rightarrow \mathrm{WP}(res := b/a, \psi_3))$$

2. For each assignment with division, add the non-zero denominator condition (Method 3):
   - $\mathrm{WP}(res := a/b, \psi_3) = (a/b \geq 1) \wedge (b \neq 0)$
   - $\mathrm{WP}(res := b/a, \psi_3) = (b/a \geq 1) \wedge (a \neq 0)$

3. Therefore:

$$\mathrm{WP}(g, \psi_3) = (a \geq b \Rightarrow (a/b \geq 1 \wedge b \neq 0)) \wedge (a < b \Rightarrow (b/a \geq 1 \wedge a \neq 0))$$

   i.e., separating the division conditions:

$$(a \geq b \Rightarrow a/b \geq 1) \wedge (a < b \Rightarrow b/a \geq 1) \wedge (a \geq b \Rightarrow b \neq 0) \wedge (a < b \Rightarrow a \neq 0).$$

Example 4: Proof of a WP Property by Induction (Method 6)

Context: Prove that for any program c and formulas P, Q,

$$\mathrm{WP}(c, P \wedge Q) = \mathrm{WP}(c, P) \wedge \mathrm{WP}(c, Q).$$

Application (sketch of proof by structural induction):

1. **Base case**: c is an assignment $x := e$.

- $\mathrm{WP}(x := e, P \wedge Q) = (P \wedge Q)[x \leftarrow e] = P[x \leftarrow e] \wedge Q[x \leftarrow e]$
- $\mathrm{WP}(x := e, P) \wedge \mathrm{WP}(x := e, Q) = P[x \leftarrow e] \wedge Q[x \leftarrow e]$
  $\rightarrow$ Equality verified.

2. **Conditional case**: $c = \text{if } B \text{ then } c_1 \text{ else } c_2$.
   - By induction hypothesis, $\mathrm{WP}(c_1, P \wedge Q) = \mathrm{WP}(c_1, P) \wedge \mathrm{WP}(c_1, Q)$ (same for c_2).
   - Then:

$$\begin{aligned}\mathrm{WP}(c, P \wedge Q) &= (B \Rightarrow \mathrm{WP}(c_1, P \wedge Q)) \wedge (\neg B \Rightarrow \mathrm{WP}(c_2, P \wedge Q)) \\ &= (B \Rightarrow (\mathrm{WP}(c_1, P) \wedge \mathrm{WP}(c_1, Q))) \wedge (\neg B \Rightarrow (\mathrm{WP}(c_2, P) \wedge \mathrm{WP}(c_2, Q))) \\ &= (B \Rightarrow \mathrm{WP}(c_1, P)) \wedge (B \Rightarrow \mathrm{WP}(c_1, Q)) \wedge (\neg B \Rightarrow \mathrm{WP}(c_2, P)) \wedge (\neg B \Rightarrow \mathrm{WP}(c_2, Q)) \\ &= \mathrm{WP}(c, P) \wedge \mathrm{WP}(c, Q)\end{aligned}$$

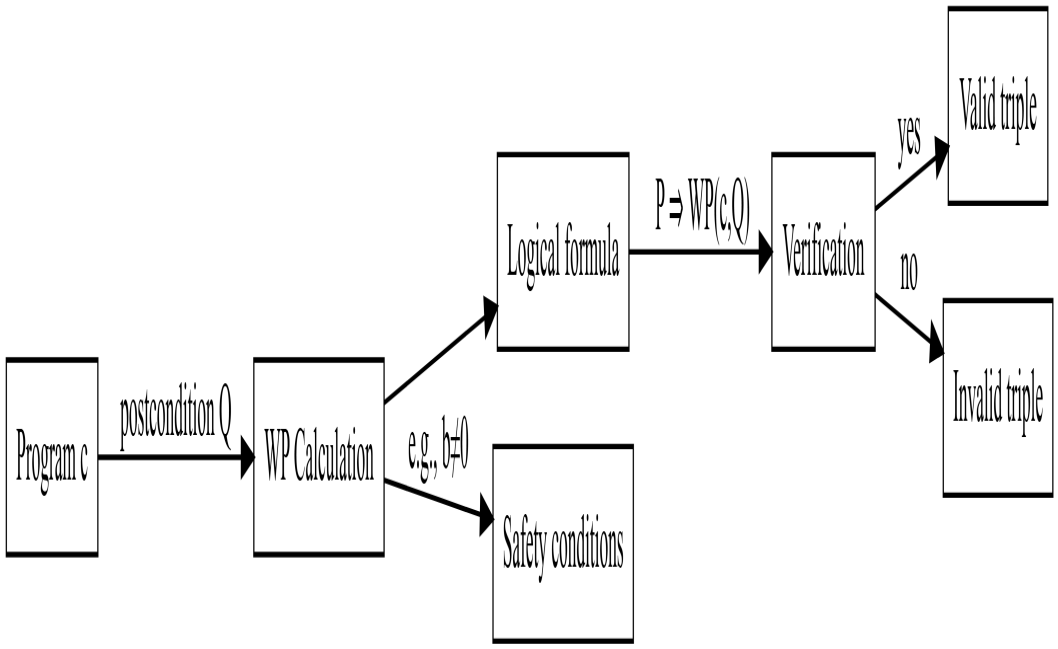
   $\rightarrow$ The property is preserved.

3. **Other cases** (sequence, loop) are treated similarly.

Synthesis

The exercises implement fundamental techniques of deductive verification:

- **Weakest precondition calculation** transforms a program verification problem into a logical problem.
- **Hoare triple validity** reduces to a logical implication, which may require adding **safety conditions** (e.g., non-division by zero).
- **Structural induction proofs** on program syntax allow establishing generic WP properties.
- These methods are the basis of formal verification tools like Why3, where WPs are generated and verified automatically.


Program c
postcondition Q
WP Calculation
Logical formula
e.g., b≠0
Safety conditions
P ⇒ WP(c,Q)
Verification
yes
no
Valid triple
Invalid triple